Django Professional Notes
Security, Model Managers, `cls`, and Django ORM

> **Purpose:** A clean, structured reference for learning Django security, Model Managers, class methods, QuerySets, and common ORM operations.

Part 1 — Django Security

Django is a security-focused web framework. It provides built-in security features and recommended settings that help protect applications from many common web security problems.

---

1. Cross-Site Scripting (XSS) Protection

What is XSS?

Cross-Site Scripting (XSS) is an attack where an attacker attempts to inject malicious HTML or JavaScript into a web page so that it executes in another user's browser.

Example:

```html
<script>
    alert("Hacked");
</script>
```

Suppose a user saves this malicious content in the database.

How does Django protect against XSS?

Django's template system normally **automatically escapes HTML special characters** when rendering variable output.

```django
{{ username }}
```

If `username` contains:

```html
<script>alert("Hacked")</script>
```

the browser receives escaped HTML instead of executable JavaScript:

```html
&lt;script&gt;alert("Hacked")&lt;/script&gt;
```

Be careful with the `safe` filter

```django
{{ username|safe }}
```

The `safe` filter tells Django not to HTML-escape that value.

Therefore:

> **Do not use `safe` on untrusted user input.**

Use it only when the content is trusted and has been properly sanitized.

Key Point

Django's automatic escaping is an important XSS defense, but insecure HTML/JavaScript handling by the developer can still introduce XSS vulnerabilities.

---

2. Cross-Site Request Forgery (CSRF) Protection

What is CSRF?

Cross-Site Request Forgery (CSRF) is an attack where an attacker attempts to make a logged-in user's browser send an unwanted state-changing request.

For example, a user is logged in to a website. An attacker may create a malicious page containing:

```html
<form action="https://example.com/change-email/" method="POST">
    <input type="hidden" name="email" value="attacker@example.com">
</form>
```

The attacker's goal is to make the victim's browser submit the request.

Django's Solution

Django uses a **CSRF token** and CSRF middleware.

Example:

```html
<form method="post">
    {% csrf_token %}

    <input type="text" name="name">

    <button type="submit">
        Submit
    </button>
</form>
```

Django's CSRF middleware validates the token.

Important

CSRF protection must be correctly configured and used for state-changing requests such as:

- `POST`
- `PUT`
- `PATCH`
- `DELETE`

---

3. SQL Injection Protection

What is SQL Injection?

SQL Injection is an attack where malicious input attempts to manipulate the structure of a SQL query.

Unsafe Example

```python
query = "SELECT * FROM users WHERE name='%s'" % username
```

Directly inserting user input into an SQL string is unsafe.

Django ORM

Django ORM normally uses parameterized queries.

```python
User.objects.filter(username=username)
```

The user input is not manually concatenated into the SQL string.

Important

Django ORM greatly reduces SQL injection risk, but it does **not** mean every SQL-related security problem is automatically solved.

Be especially careful when using:

- Raw SQL
- Dynamic SQL

- Unsafe query construction

Example:

```python
from django.db import connection
```

When using raw SQL, use proper parameterization.

Remember

> **Using ORM does not mean every SQL security problem is automatically solved.**

---

4. Clickjacking Protection

What is Clickjacking?

Clickjacking is an attack where an attacker places a trusted website inside another page, often through an `iframe`, and tricks the user into clicking something without realizing what they are actually clicking.

Django Protection

Django can use the `X-Frame-Options` header for clickjacking protection.

`DENY`

```python
X_FRAME_OPTIONS = "DENY"
```

The site cannot be loaded inside an iframe.

`SAMEORIGIN`

```python
X_FRAME_OPTIONS = "SAMEORIGIN"
```

```

The page may be loaded in an iframe when the embedding page has the same origin.

### Which should you use?

Choose according to your application's requirements.

If your application does not need iframe embedding, you can generally use:

```python
X_FRAME_OPTIONS = "DENY"
```

---

5. SSL / HTTPS

What is HTTPS?

HTTPS encrypts communication between the browser and the server.

This makes it much harder for an attacker on the network to read or modify transmitted data.

Django Settings

To redirect HTTP requests to HTTPS:

```python
SECURE_SSL_REDIRECT = True
```

For production applications, secure cookies are also important:

```python
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```
```

````
### Important

`SECURE_SSL_REDIRECT` alone does not provide complete HTTPS security.

Production deployment also requires correct:

- HTTPS configuration
- Proxy/load-balancer configuration
- TLS/SSL configuration
- Cookie security

---

# 6. Host Header Validation

## What is the Host Header?

HTTP requests contain a `Host` header.

Example:

```http
Host: example.com
```

If an application accepts arbitrary hosts, certain Host Header-related attacks may become possible.

## Django Protection

Django uses `ALLOWED_HOSTS`.

```python
ALLOWED_HOSTS = [
    "example.com",
    "www.example.com",
]
```

This tells Django which host names are allowed.
````

Development

A development configuration may use:

```python
ALLOWED_HOSTS = [
    "localhost",
    "127.0.0.1",
]
```

Production

Use the specific domains required by your application.

---

7. Referrer Policy

What is a Referrer?

When a browser moves from one webpage to another resource or website, it may send information about the previous page through the `Referer` header.

That information can sometimes expose sensitive URL details.

Django Setting

```python
SECURE_REFERRER_POLICY = "same-origin"
```

This controls how referrer information is sent.

Another possible policy is:

```python
SECURE_REFERRER_POLICY = "strict-origin-when-cross-origin"
```

Choose the policy according to your application's security and privacy requirements.

---

8. Session Security

After login, Django can use sessions to maintain a user's authentication state.

Securing session cookies is therefore important.

Important Settings

8.1 Secure Cookie

```python
SESSION_COOKIE_SECURE = True
```

The session cookie is sent only over HTTPS connections.

8.2 HttpOnly Cookie

```python
SESSION_COOKIE_HTTPONLY = True
```

Normal client-side JavaScript cannot access the session cookie.

This helps reduce some client-side cookie theft risks.

8.3 Expire Session When Browser Closes

```python
SESSION_EXPIRE_AT_BROWSER_CLOSE = True
```

This configures session-cookie expiration behavior when the browser session ends.

````
### Important

This setting does not solve every authentication/security problem.

Good authentication/session security also requires:

- Strong passwords
- HTTPS
- Secure cookies
- Appropriate session management

---

# 9. User-Uploaded Content Security

Allowing users to upload files introduces security risks.

Possible risks include:

- Malicious files
- Very large files
- Unexpected file types
- Dangerous content
- Storage abuse
- Filename-related problems

## 9.1 File Type Validation

Django provides validators such as `FileExtensionValidator`.

```python
from django.core.validators import FileExtensionValidator

file = models.FileField(
    upload_to="documents/",
    validators=[
        FileExtensionValidator(
            allowed_extensions=["jpg", "png", "pdf"]
        )
    ]
````
````

```
)
```

Important

Checking only the file extension is **not always sufficient** because an attacker may rename a malicious file.

Sensitive applications should validate file content/type more carefully.

9.2 File Size Limits

Do not allow unlimited file uploads.

Configure appropriate upload limits at:

- Application level
- Web server/infrastructure level

9.3 Media Files

User-uploaded files are commonly stored as media.

Example:

```text
media/
    images/
    documents/
    uploads/
```

In production, the architecture for serving user-uploaded content should be designed carefully.

> User-uploaded files must never be allowed to execute as server-side code.

---

10. Django Security Middleware

Django's security features rely partly on middleware.

Example:

```python
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
]
```

Important Middleware

`SecurityMiddleware`

Helps apply security-related HTTP behavior and security settings.

`CsrfViewMiddleware`

Provides important CSRF protection.

---

11. Additional Security Settings

11.1 HSTS

HTTP Strict Transport Security (HSTS) tells browsers to use HTTPS more strictly.

```python
SECURE_HSTS_SECONDS = 31536000
```

Depending on the deployment requirements:

```python
```

````
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
```

> **Warning:** Enable HSTS only after confirming that the domain and HTTPS configuration are working correctly.

## 11.2 Secure CSRF Cookie

```python
CSRF_COOKIE_SECURE = True
```

This helps ensure the CSRF cookie is sent over HTTPS.

## 11.3 Content-Type Sniffing Protection

```python
SECURE_CONTENT_TYPE_NOSNIFF = True
```

This helps limit browser MIME-type sniffing behavior.

---

# 12. Production Security Checklist

Before deploying a Django project to production, check:

```text
☑ DEBUG = False

☑ ALLOWED_HOSTS properly configured

☑ HTTPS enabled

☑ SECURE_SSL_REDIRECT configured where appropriate

☑ SESSION_COOKIE_SECURE = True

☑ CSRF_COOKIE_SECURE = True
````

```
☑ SESSION_COOKIE_HTTPONLY = True

☑ CSRF protection enabled

☑ X_FRAME_OPTIONS configured

☑ SECURE_CONTENT_TYPE_NOSNIFF = True

☑ Appropriate SECURE_REFERRER_POLICY

☑ HSTS configured carefully

☑ Strong authentication

☑ User upload validation

☑ File size limits

☑ Secrets kept outside source code

☑ Dependencies regularly updated
```

---

13. Security Quick Revision

Security Problem	Django Protection
XSS	Template auto-escaping
CSRF	CSRF token + middleware
SQL Injection	ORM / parameterized queries
Clickjacking	`X_FRAME_OPTIONS`
HTTPS	`SECURE_SSL_REDIRECT` + secure deployment
Host Header Attack	`ALLOWED_HOSTS`
Referrer Leakage	`SECURE_REFERRER_POLICY`
Session Security	Secure + HttpOnly cookies
Unsafe Upload	Validation + size limits + safe storage
MIME Sniffing	`SECURE_CONTENT_TYPE_NOSNIFF`
HSTS	`SECURE_HSTS_*` settings

Part 2 — Django Model Managers and Related Objects

14. What is a Model Manager?

A **Django Model Manager** is a class that provides an interface for database queries.

Every Django model has a default manager, commonly accessed through:

```python
objects
```

Example:

```python
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.FloatField()
```

Then:

```python
Product.objects.all()
```

returns all products.

```python
Product.objects.filter(price__gt=100)
```

returns products whose price is greater than 100.

---

15. Custom Model Manager

You can create a custom manager and add reusable query methods.

```python
class ProductManager(models.Manager):
    def expensive_products(self):
        return self.filter(price__gt=500)
```

Attach it to the model:

```python
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.FloatField()

    objects = ProductManager()
```

Use it:

```python
Product.objects.expensive_products()
```

Why use a Custom Manager?

It helps you:

- Reuse query logic
- Keep query-related code organized
- Create readable model APIs
- Avoid repeating common filters

---

16. Model Relationships and Related Objects

The term **"nodes"** is not a standard Django Model Manager concept.

A clearer way to understand the example is through **model instances and related objects**.

Suppose we have:

```text
Category
   │
   └── Product
          │
          └── Variant
```

Example:

```python
class Category(models.Model):
    title = models.CharField(max_length=100)


class Product(models.Model):
    category = models.ForeignKey(
        Category,
        on_delete=models.CASCADE
    )
    name = models.CharField(max_length=100)
    price = models.FloatField()
```

If:

```python
cat = Category.objects.get(id=1)
```

you can access related products:

```python
products_in_cat = cat.product_set.all()
```

Here:

- `Category.objects` → Manager
- `cat` → Model instance
- `cat.product_set` → Reverse relationship manager
- `.all()` → QuerySet

Key Idea

> **Manager handles queries; model instances represent database records; related managers provide access to related records.**

Part 3 — Python `cls` and Django Managers

17. What does `cls` mean?

`cls` refers to the **class itself**, not an instance.

It is commonly used as the first parameter of a Python `@classmethod`.

Compare:

```text
self → instance
cls  → class
```

Example:

```python
class MyClass:
    count = 0

    @classmethod
    def increment(cls):
        cls.count += 1
```

Call it directly from the class:

```python
MyClass.increment()

print(MyClass.count)
# Output: 1
```

No object was created.

Here:

```text
cls = MyClass
```

---

18. Django's `objects`

In a Django model:

```python
Product.objects
```

`objects` is a **Model Manager**.

It provides methods for querying the database.

Examples:

```python
Product.objects.all()
```

```python
Product.objects.filter(price__lt=100)
```

```python
Product.objects.get(pk=1)
```

---

19. Class Method + Django Manager

A class method can use the model's manager through `cls`.

Example:

```python
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.FloatField()

    @classmethod
    def expensive_items(cls):
        return cls.objects.filter(price__gt=500)
```

Use it:

```python
expensive = Product.expensive_items()
```

What happens internally?

```text
Product.expensive_items()
        ↓
cls = Product
        ↓
cls.objects
        ↓
Product.objects
        ↓
filter(price__gt=500)
        ↓
```

```
QuerySet
        ↓
Database
        ↓
Results
```

Easy Memory Trick

```text
cls     → class
objects → manager
filter  → database query
```

---

20. `cls → objects → QuerySet → Database`

A useful mental model is:

```text
        Product Class
             |
             | cls
             ▼
        cls.objects
             |
             | Manager
             ▼
          QuerySet
      ┌──────┼──────┐
      |      |      |
     all() filter() get()
             |
             ▼
         Database
             |
             ▼
          Results
```

Example:

```python
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.FloatField()

    @classmethod
    def expensive_items(cls):
        return cls.objects.filter(price__gt=500)
```

Conceptually:

```python
Product.expensive_items()
```

becomes:

```python
Product.objects.filter(price__gt=500)
```

Part 4 — Django ORM A–Z Cheat Sheet

21. Basic QuerySet Operations

21.1 Get All Records

```python
students = Student.objects.all()
```

21.2 Filter

```python
students = Student.objects.filter(
```

```
    department="CMT"
)
```

21.3 Multiple Conditions

```python
students = Student.objects.filter(
    department="CMT",
    age=20
)
```

21.4 Exclude

```python
students = Student.objects.exclude(
    department="BBA"
)
```

21.5 Get One Object

```python
student = Student.objects.get(pk=1)
```

> `get()` is intended to return exactly one object. It raises an exception if no matching object exists or multiple objects match.

21.6 First Object

```python
student = Student.objects.first()
```

21.7 Last Object

```python
student = Student.objects.last()
```

---

22. Ordering

22.1 Ascending Order

```python
students = Student.objects.order_by("name")
```

22.2 Descending Order

```python
students = Student.objects.order_by("-name")
```

22.3 Random Order

```python
students = Student.objects.order_by("?")
```

> Random ordering can be expensive on some databases, especially with large tables.

22.4 Reverse Existing Ordering

```python
students = Student.objects.order_by("pk").reverse()
```

---

23. Counting and Selecting Fields

23.1 Count

```python
total = Student.objects.count()
```

23.2 `values()`

Returns dictionaries containing selected fields.

```python
students = Student.objects.values(
    "name",
    "department"
)
```

Conceptually:

```python
[
    {"name": "A", "department": "CMT"},
    {"name": "B", "department": "BBA"},
]
```

23.3 `values_list()`

Returns tuples by default.

```python
students = Student.objects.values_list(
    "name",
    "department"
)
```

---

24. Creating Records

24.1 `create()`

```python
Student.objects.create(
    name="Faruk",
```

```
    department="CMT",
    roll=101
)
```

24.2 `get_or_create()`

```python
student, created = Student.objects.get_or_create(
    name="Faruk",
    department="CMT"
)
```

The result contains:

```text
student → object
created → True/False
```

---

25. Updating Records

25.1 `update()`

```python
Student.objects.filter(pk=1).update(
    name="Ahmed"
)
```

25.2 `update_or_create()`

```python
student, created = Student.objects.update_or_create(
    pk=1,
    defaults={
        "name": "Ahmed"
    }
```

```
)
```

---

26. Deleting Records

26.1 Delete Using a QuerySet

```python
Student.objects.filter(pk=1).delete()
```

26.2 Delete a Single Object

```python
student = Student.objects.get(pk=1)
student.delete()
```

---

27. Bulk Operations

27.1 `bulk_create()`

Create multiple objects efficiently:

```python
Student.objects.bulk_create([
    Student(name="A"),
    Student(name="B"),
    Student(name="C"),
])
```

27.2 `bulk_update()`

```python
students = Student.objects.filter(
    department="CMT"
```

```
)

for student in students:
    student.department = "BBA"

Student.objects.bulk_update(
    students,
    ["department"]
)
```

27.3 `in_bulk()`

```python
students = Student.objects.in_bulk([1, 2, 3])
```

---

28. Field Lookups

Django provides powerful lookup expressions.

28.1 Range

```python
students = Student.objects.filter(
    roll__range=(101, 110)
)
```

28.2 `IN`

```python
students = Student.objects.filter(
    id__in=[1, 2, 3]
)
```

28.3 Less Than

```python
students = Student.objects.filter(
    age__lt=20
)
```

28.4 Greater Than

```python
students = Student.objects.filter(
    age__gt=20
)
```

28.5 Less Than or Equal

```python
students = Student.objects.filter(
    age__lte=20
)
```

28.6 Greater Than or Equal

```python
students = Student.objects.filter(
    age__gte=20
)
```

---

29. Text Lookups

29.1 Exact Match

```python
students = Student.objects.filter(
    name__exact="Faruk"
)
```

29.2 Case-Insensitive Exact Match

```python
students = Student.objects.filter(
    name__iexact="faruk"
)
```

29.3 Contains

```python
students = Student.objects.filter(
    name__contains="aru"
)
```

29.4 Case-Insensitive Contains

```python
students = Student.objects.filter(
    name__icontains="aru"
)
```

29.5 Starts With

```python
students = Student.objects.filter(
    name__startswith="F"
)
```

29.6 Ends With

```python
students = Student.objects.filter(
    name__endswith="k"
)
```

29.7 Regular Expression

```python
students = Student.objects.filter(
    name__regex=r"^F"
)
```

---

30. Date-Based Queries

30.1 `dates()`

```python
students = Student.objects.dates(
    "passed_in_year",
    "year"
)
```

30.2 Latest

```python
student = Student.objects.latest(
    "created_at"
)
```

30.3 Earliest

```python
student = Student.objects.earliest(
    "created_at"
)
```

> `latest()` and `earliest()` require an appropriate date/datetime field and ordering configuration.

---

31. Aggregation

Django ORM supports aggregate calculations.

Import:

```python
from django.db.models import (
    Avg,
    Sum,
    Max,
    Min,
    Count,
)
```

Average

```python
Student.objects.aggregate(
    Avg("marks")
)
```

Sum

```python
Student.objects.aggregate(
    Sum("marks")
)
```

Maximum

```python
Student.objects.aggregate(
    Max("marks")
)
```

Minimum

```python
Student.objects.aggregate(
    Min("marks")
)
```

Count

```python
Student.objects.aggregate(
    Count("id")
)
```

Concept

```text
QuerySet
   |
   ├── Avg()
   ├── Sum()
   ├── Max()
   ├── Min()
   └── Count()
   |
   ▼
Aggregate Result
```

---

32. `annotate()`

`annotate()` adds calculated information to each result/group.

Example:

```python
from django.db.models import Count
```

```
students = (
    Student.objects
    .values("department")
    .annotate(total=Count("id"))
)
```

This can produce results conceptually like:

```text
CMT → 50
BBA → 35
CSE → 60
```

---

33. Using a Specific Database

If a Django project has multiple databases, you can select one with `using()`.

```python
students = Student.objects.using("default")
```

The string identifies the configured database alias.

---

34. Raw SQL

Django also provides a way to execute raw SQL through a model.

Example:

```python
students = Student.objects.raw(
    "SELECT * FROM home_student"
)
```

```

### Security Warning

Raw SQL should be used carefully.

Never build SQL queries by directly concatenating untrusted user input.

Prefer parameterized queries when raw SQL is necessary.

---

# Part 5 — Interview Revision

## 35. What is XSS?

XSS is an attack where malicious script or HTML is injected into a webpage and executed in another user's browser.

## 36. How does Django protect against XSS?

Django templates automatically escape variable output by default.

## 37. What is CSRF?

CSRF is an attack where an attacker attempts to make an authenticated user's browser send an unwanted state-changing request.

## 38. How does Django protect against CSRF?

Django uses CSRF tokens and CSRF middleware.

## 39. What is SQL Injection?

SQL Injection is an attack where malicious input manipulates the structure of a SQL query.

## 40. Does Django ORM help prevent SQL Injection?

Yes. Django ORM normally uses parameterized queries, but unsafe raw SQL can still introduce vulnerabilities.
```

41. What is Clickjacking?

Clickjacking tricks users into interacting with a webpage or action through a deceptive frame or interface.

42. What is `ALLOWED_HOSTS`?

`ALLOWED_HOSTS` defines which host/domain names Django will accept for incoming HTTP requests.

43. What does `SESSION_COOKIE_SECURE` do?

It makes the session cookie available only over HTTPS connections.

44. Why is `SESSION_COOKIE_HTTPONLY` useful?

It prevents normal client-side JavaScript from reading the session cookie.

45. What is a Django Model Manager?

A Model Manager provides the interface through which Django performs database queries for a model.

46. What is `objects`?

`objects` is the commonly used default Model Manager.

Example:

```python
Product.objects.all()
```

47. What is `cls`?

`cls` refers to the class itself and is commonly used as the first parameter of a `@classmethod`.

48. What is a QuerySet?

A QuerySet represents a collection of database queries/results and provides methods such as:

```python
all()
filter()
exclude()
get()
order_by()
```

49. What is the difference between `self` and `cls`?

```text
self → current object/instance
cls  → current class
```

50. What is the difference between Manager and QuerySet?

```text
Manager
   ↓
Creates/starts database queries
   ↓
QuerySet
   ↓
Represents a database query/result collection
```

Part 6 — Final Revision Summary

Django Security

```text
XSS
 ↓
Automatic Template Escaping
```

```
CSRF
 ↓
CSRF Token + Middleware

SQL Injection
 ↓
ORM + Parameterized Queries

Clickjacking
 ↓
X-Frame-Options

Host Validation
 ↓
ALLOWED_HOSTS

Session Security
 ↓
Secure + HttpOnly Cookies

HTTPS Security
 ↓
SSL/TLS + Security Settings

File Upload Security
 ↓
Validation + Size Limits + Safe Storage
```

Django ORM

```text
Model
  ↓
Manager
  ↓
QuerySet
  ↓
Database
  ↓
```

```
Result
```

`cls` + Manager

```text
Class Method
     ↓
    cls
     ↓
cls.objects
     ↓
  Manager
     ↓
 QuerySet
     ↓
 Database
     ↓
  Result
```

Essential ORM Methods

```text
all()
filter()
exclude()
get()

first()
last()
order_by()
count()

values()
values_list()

create()
get_or_create()

update()
```

````
update_or_create()

delete()

bulk_create()
bulk_update()
in_bulk()

aggregate()
annotate()

using()
raw()
```

---

# Final Takeaway

Django provides strong built-in security mechanisms, but a Django application is **not automatically 100% secure**.

Application security depends on the complete system:

```text
Django Security Features
        +
Secure Application Code
        +
Authentication & Authorization
        +
Safe File Handling
        +
Secure Database Usage
        +
Updated Dependencies
        +
Correct Deployment Configuration
        +
Server / Infrastructure Security
        =
````

````
Secure Django Application
```

For ORM work, remember the core relationship:

```text
Model
  ↓
Manager (`objects`)
  ↓
QuerySet
  ↓
Database
  ↓
Result
```

And for class methods:

```text
self → instance
cls  → class
```

The most important concepts to master are:

- Django Security
- XSS
- CSRF
- SQL Injection
- Clickjacking
- HTTPS
- `ALLOWED_HOSTS`
- Secure Cookies
- File Upload Security
- Model Managers
- Custom Managers
- `cls`
- QuerySets
- Field Lookups
- Aggregation

```
    - Annotation
    - Bulk Operations
    - Raw SQL
```